

Desenvolvimento de uma API REST para Gestão de Pedidos Locais: uma aplicação prática de Sistemas Distribuídos

Curso de Ciência da Computação — Ânima
Disciplina: Sistemas Distribuídos e Mobile — Avaliação A3 (2026/01)

Integrantes: *(preencher com os nomes do grupo)* **Contato:** *(e-mail do grupo)*

Observação para a equipe: este documento é o conteúdo do artigo, redigido de acordo com o sistema realmente implementado. Copie cada seção para dentro do template oficial do Google Docs, preservando a formatação exigida pelos professores. Os trechos em itálico entre parênteses devem ser preenchidos/ajustados.

Resumo

Este trabalho apresenta o desenvolvimento do **PratoFácil**, uma aplicação web e **API REST** construída em **Java com Spring Boot 4**, cujo objetivo é auxiliar pequenos empreendedores do ramo alimentício na gestão do ciclo de vida de seus pedidos. O sistema centraliza o cadastro do cardápio, a realização de pedidos pelos clientes e o acompanhamento do status de entrega, aplicando na prática os conceitos da disciplina de Sistemas Distribuídos: arquitetura cliente-servidor, comunicação por **HTTP**, **API REST**, **middleware**, **transparência**, persistência de dados e **serviços em nuvem**. A solução expõe o mesmo núcleo de negócio por duas interfaces — uma API REST que troca dados em JSON (consumível por qualquer cliente HTTP, como o Postman) e uma interface web responsiva (Thymeleaf) — e adota autenticação por papéis (empreendedor e cliente). Os testes, automatizados com MockMvc e manuais via requisições HTTP, validaram o comportamento dos endpoints e o tratamento padronizado de erros.

Palavras-chave: API REST; Spring Boot; Sistemas Distribuídos; HTTP; Gestão de Pedidos.

Abstract

This paper presents **PratoFácil**, a web application and **REST API** built with **Java and Spring Boot 4** to help small food entrepreneurs manage the life cycle of their orders. The system centralizes menu registration, order placement by customers and delivery status tracking, applying core Distributed Systems concepts: client-server architecture, **HTTP** communication, **REST API**, **middleware**, **transparency**, data persistence and **cloud services**. The solution exposes the same business core through two interfaces — a JSON REST API (consumable by any HTTP client, such as Postman) and a responsive web interface (Thymeleaf) — and adopts role-based authentication (entrepreneur and customer). Tests, automated with MockMvc and manual via HTTP requests, validated endpoint behavior and standardized error handling.

Keywords: REST API; Spring Boot; Distributed Systems; HTTP; Order Management.

1. Introdução

Com o crescimento dos pequenos empreendimentos no setor alimentício, muitos processos de gerenciamento de pedidos ainda são realizados de forma manual, principalmente por aplicativos de men-

sagens e anotações informais. Esse modelo favorece a perda de informações, erros nos pedidos e dificulta o acompanhamento e a escalabilidade do negócio.

Diante desse contexto, este trabalho apresenta o **PratoFácil**, um sistema de gestão de pedidos desenvolvido como uma **API REST** em **Java/Spring Boot**, acompanhado de uma interface web. O objetivo é centralizar e organizar o ciclo de vida do pedido — do cadastro do cardápio até a entrega — em uma arquitetura distribuída e desacoplada, conectando a teoria estudada na disciplina de Sistemas Distribuídos a um problema real e observável. O sistema é estruturado como um pequeno **marketplace**: cada loja (empreendedor) possui seu próprio cardápio, e o cliente, ao acessar a plataforma, escolhe em qual loja deseja realizar o pedido.

O escopo foi deliberadamente **reduzido e bem trabalhado**, conforme orientação da avaliação, concentrando-se em três requisitos funcionais:

- **RF01:** listar o cardápio disponível;
- **RF02:** permitir que o cliente realize um pedido via requisição HTTP;
- **RF03:** permitir que o empreendedor atualize o status do pedido (*Em preparo, Saiu para entrega, Entregue*).

2. Fundamentação Teórica e Conceitos Aplicados

Esta seção relaciona os conceitos teóricos da disciplina com os pontos do projeto em que eles aparecem.

2.1 Sistemas Distribuídos e Transparência

Um sistema distribuído é composto por componentes que se comunicam por uma rede para executar tarefas de forma integrada. No PratoFácil, a separação entre **cliente** (navegador ou Postman), **servidor de aplicação** (API Spring Boot) e **banco de dados** caracteriza essa distribuição. O cliente realiza uma requisição HTTP e **não precisa saber** onde o servidor está hospedado, em qual linguagem foi escrito, nem onde os dados são armazenados — caracterizando a **transparência de localização e de acesso**. Essa abstração é reforçada pela possibilidade de hospedar o banco de dados em nuvem, separado da aplicação (Seção 2.4).

2.2 Middleware e Webservices

O **Spring Boot** atua como o **middleware** da aplicação: recebe as requisições HTTP, aplica as regras de negócio (cálculo do valor total do pedido, validações, controle de status) e intermedeia o acesso ao banco de dados por meio do **Spring Data JPA**. Os serviços são disponibilizados como **webservices REST**, permitindo que diferentes clientes consumam os mesmos recursos.

2.3 Protocolo HTTP e API REST

A comunicação segue o estilo arquitetural **REST** (Representational State Transfer), padronizando a interação cliente-servidor sobre **HTTP**. São utilizados os verbos:

- **GET** para consultas (listar o cardápio — RF01);
- **POST** para criação (cadastrar prato, criar pedido — RF02);
- **PUT** para atualização (editar prato, atualizar status do pedido — RF03);
- **DELETE** para remoção (excluir prato).

Os dados trafegam em **JSON**, formato amplamente adotado por sua simplicidade e interoperabilidade. As respostas utilizam os **códigos de status HTTP** de forma semântica (200, 201, 204, 400, 401, 404, 409), e os erros retornam um corpo JSON padronizado, sem expor detalhes internos (*stack trace*).

2.4 Serviços em Nuvem e Banco de Dados

A persistência é feita com **Spring Data JPA** sobre um banco relacional. O projeto foi estruturado com **perfis de execução**: no perfil de desenvolvimento (**dev**) utiliza-se **H2 em arquivo** (os dados persistem entre reinícios do servidor; nos testes automatizados usa-se um H2 em memória isolado), e no perfil de produção (**prod**) a aplicação utiliza um **PostgreSQL hospedado em nuvem** (provedor **Render**), com as credenciais fornecidas por variáveis de ambiente. A conexão foi **validada em execução**: a aplicação conectou-se ao PostgreSQL na nuvem (versão 18), criou o esquema automaticamente e persistiu/recuperou dados com sucesso, sem qualquer alteração no código — apenas a troca do perfil. Isso demonstra, na prática, os conceitos de **banco de dados em nuvem** e de **transparência de localização**: a aplicação não muda — apenas o endereço do banco — ao migrar do ambiente local para o ambiente em nuvem.

2.5 Segurança e Controle de Acesso

Como a plataforma envolve dois perfis de usuário, foi adotado o **Spring Security** com autenticação baseada em formulário e em **HTTP Basic** (para testes da API), além de **autorização por papéis**: **ADMIN** (empreendedor) e **CLIENTE**. As senhas são armazenadas com **hash BCrypt**, e cada cliente acessa somente os próprios pedidos.

2.6 Integração com Webservice Externo e Comunicação Assíncrona (Webhook)

O fluxo de **pagamento** é o exemplo mais direto de **comunicação entre sistemas distribuídos** no projeto: a aplicação não processa o pagamento sozinha — ela se **integra a um serviço de terceiros**, o gateway **Asaas** (em ambiente *sandbox*). Essa integração ocorre em duas direções, ilustrando dois paradigmas de comunicação:

- **Comunicação síncrona (aplicação → Asaas)**: ao finalizar o pedido, o servidor consome a **API REST do Asaas** via HTTP (cliente **RestClient** do Spring), criando o cliente e a cobrança **PIX** e obtendo o **QR Code**. Aqui o PratoFácil atua como **cliente** de outro webservice, evidenciando a interoperabilidade entre sistemas heterogêneos por meio de contratos HTTP/JSON.
- **Comunicação assíncrona (Asaas → aplicação) via *webhook***: quando o pagamento é confirmado, o Asaas envia, por conta própria, uma requisição HTTP para o *endpoint/webhooks/asaas* da aplicação, que então atualiza o status do pedido para *Pago*. Esse padrão **orientado a eventos** (*event-driven*) desacopla os sistemas no tempo: a aplicação não fica “perguntando” se o pagamento ocorreu (*polling*) — ela é **notificada** quando o evento acontece. Opcionalmente, valida-se um *token* no cabeçalho da requisição para garantir a autenticidade da chamada.

A integração foi desenhada para **degradar com elegância**: sem a chave de API configurada, a etapa de pagamento opera como uma **simulação** local, mantendo a aplicação funcional sem a dependência externa — uma decisão útil tanto para desenvolvimento quanto para a demonstração acadêmica.

3. Metodologia e Desenvolvimento

3.1 Tecnologias utilizadas

- **Linguagem:** Java 21
- **Framework:** Spring Boot 4 (Spring Web MVC, Spring Data JPA, Spring Security, Thymeleaf)
- **Banco de dados:** H2 em arquivo (perfil dev) e PostgreSQL em nuvem/Render (perfil prod)
- **Pagamentos:** integração com o gateway **Asaas** (sandbox) — cobrança PIX + webhook
- **Documentação da API:** springdoc-openapi (Swagger UI / OpenAPI 3)
- **Containerização e deploy:** Docker (build multi-stage) e Render (`render.yaml`)
- **Build e dependências:** Maven (wrapper `mvnw`)
- **Testes:** JUnit 5 + Spring MockMvc; Postman/cURL para testes manuais
- **Versionamento:** Git/GitHub

3.2 Arquitetura

A aplicação segue o padrão **MVC** com uma **camada de serviço** que isola as regras de negócio dos controladores. O mesmo núcleo (serviços → repositórios → banco) é exposto por dois tipos de controlador:

- **Controladores REST** (`/api/...`): trocam JSON, demonstrando os conceitos de SD;
- **Controladores Web** (Thymeleaf): renderizam páginas HTML responsivas.

Organização do código:

```
controller/ -> REST (/api), Web (Thymeleaf), webhook + tratamento global de erros
service/    -> regras de negocio (Cardapio, Pedido, Usuario, Pagamento, Asaas)
repository/ -> repositórios Spring Data JPA
model/      -> entidades JPA (Loja, Cardapio, Pedido, ItemPedido, Usuario) e enums
              (Status, Role, Categoria, StatusPagamento, TipoPagamento)
exception/  -> excecoes de dominio (RecursoNaoEncontrado, RegraNegocio, SessaoInvalida)
config/     -> seguranca, documentacao OpenAPI e carga inicial (lojas + cardapios)
```

Fluxo da aplicação:

```
Cliente (navegador / Postman / app) -> HTTP -> API REST Spring Boot
                                     -> Camada de Servico (regras de negocio) -> Spring Data JPA -> Banco de Dados
```

Decisão de projeto relevante: o **valor total do pedido é sempre calculado no servidor** a partir dos identificadores dos pratos enviados pelo cliente — o cliente nunca informa preços, evitando inconsistências e fraudes.

Modelo de marketplace (múltiplas lojas). O domínio possui a entidade Loja: cada usuário ADMIN é dono de uma loja, e tanto os pratos quanto os pedidos são vinculados a uma loja. Ao entrar, o cliente vê a **vitrine de lojas** e escolhe onde pedir; cada lojista gerencia **apenas a própria loja** (cardápio, pedidos e indicadores). Esse isolamento por loja é uma forma simples de **multitenancy**, em que vários estabelecimentos compartilham a mesma aplicação e o mesmo banco, porém com os dados logicamente separados — reforçando a noção de serviços compartilhados típica de sistemas distribuídos.

3.3 Endpoints da API REST

Base: <http://localhost:8080/api>

Método	Rota	Descrição	Respostas	Acesso
GET	/api/pratos	Lista o cardápio (RF01)	200	Público
GET	/api/pratos/{id}	Busca um prato	200 / 404	Público
POST	/api/pratos	Cadastra um prato	201 / 400 / 401	ADMIN
PUT	/api/pratos/{id}	Atualiza um prato	200 / 400 / 404	ADMIN
DELETE	/api/pratos/{id}	Remove um prato	204 / 404 / 409	ADMIN
GET	/api/pedidos	Lista pedidos	200	Autenticado
GET	/api/pedidos/{id}	Busca um pedido	200 / 404	Autenticado
POST	/api/pedidos	Cria um pedido (RF02)	201 / 400 / 401	CLIENTE
PUT	/api/pedidos/{id}/status	Atualiza o status (RF03)	200 / 404	ADMIN

A documentação interativa fica disponível em </swagger-ui.html> (contrato OpenAPI em </v3/api-docs>).

3.4 Interface Web

Além da API, o sistema possui páginas Thymeleaf responsivas: cardápio **organizado por categorias**, com **seletor de quantidade (+/-)** e **carrinho** (total calculado ao vivo); cadastro/login de cliente; **acompanhamento do pedido com linha do tempo de status**; painel do empreendedor com gestão de status; e um **dashboard** com indicadores (faturamento, pedidos, clientes e contagem por status). Cada item do pedido guarda a sua quantidade (entidade `ItemPedido`), e o valor total considera preço $\times$ quantidade. Após o pedido, há uma **tela de pagamento** com escolha de método (**PIX**, exibindo o QR Code real obtido do Asaas, ou **cartão de crédito**) e uma **tela de confirmação** do pagamento.

4. Documentação de Testes

4.1 Métodos

A API foi testada de duas formas: (a) **testes de integração automatizados** com Spring MockMvc, que sobem o contexto da aplicação e exercem os endpoints ponta a ponta; e (b) **testes manuais** com requisições HTTP (cURL/Postman) sobre a aplicação em execução.

4.2 Dados coletados

Os testes automatizados (todos aprovados) cobrem os principais cenários:

Cenário	Esperado	Resultado
Listar cardápio sem autenticação	200	OK
Criar prato sem autenticação	401	OK
Criar prato como ADMIN	201	OK
Criar prato sem nome (dados inválidos)	400	OK
Buscar prato inexistente	404	OK
Cliente cria pedido (total calculado no servidor)	201, EM_PREPARO	OK

Os testes manuais confirmaram ainda: criação de pedido pela interface web, isolamento de acesso (um cliente não visualiza pedidos de outro — retorno 404), atualização de status pelo empreendedor, e o retorno **409 Conflict** ao tentar remover um prato vinculado a um pedido existente.

Por fim, a aplicação foi executada no perfil **prod** conectada ao **PostgreSQL hospedado no Render**, validando a persistência em nuvem: criação de um prato (HTTP 201) e posterior leitura (HTTP 200) confirmaram que os dados foram gravados e recuperados do banco remoto.

4.3 Ajustes realizados durante o desenvolvimento

- **Serialização JSON:** a entidade **Pedido** referencia o **Usuario** cliente; para não expor dados sensíveis (incluindo o hash da senha) na resposta da API, o campo foi anotado com **@JsonIgnore**.
- **Consistência de tipos:** os identificadores das entidades foram padronizados como **Long**, eliminando conversões redundantes e potenciais inconsistências.
- **Vínculo de itens ao pedido:** corrigiu-se o nome do campo do formulário para que os itens selecionados fossem corretamente vinculados ao pedido.
- **Tratamento de erros centralizado:** um *handler* global passou a padronizar as respostas de erro (400/401/404/409) em JSON limpo.

5. Discussão

A aplicação prática dos conceitos de Sistemas Distribuídos permitiu compreender, de forma concreta, a comunicação entre componentes desacoplados. A separação entre cliente, API e banco de dados favorece a **escalabilidade** e a **manutenibilidade**: a mesma API pode atender simultaneamente a um navegador, ao Postman ou a um futuro aplicativo móvel, sem alterações no servidor. A introdução de uma camada de serviço e de exceções de domínio tornou o código mais coeso e o tratamento de erros mais previsível.

Como simplificação consciente de escopo acadêmico, a proteção contra CSRF foi desabilitada para facilitar o consumo da API por clientes HTTP e o uso do console H2; em um cenário de produção, recomenda-se reavaliá-la para os formulários web.

Limitações e escalabilidade. Dimensionar o sistema honestamente também faz parte do exercício. As telas de listagem (pedidos do cliente, pedidos da loja e dashboard) carregam todos os registros de uma vez — adequado ao volume acadêmico, mas que pediria **paginação** (**Pageable**) à medida que o número de pedidos crescesse. A criação do pedido faz uma chamada **síncrona** ao gateway de pagamento, o que acopla a latência do pedido à resposta externa; uma evolução natural seria torná-la **assíncrona**. Por fim, a hospedagem em camada gratuita (CPU/memória reduzidas e *cold start* após inatividade) limita a vazão. Nenhum desses pontos compromete o objetivo do trabalho, mas reconhecê-los demonstra a compreensão de onde a arquitetura escala e onde não.

6. Considerações Finais

O desenvolvimento do PratoFácil consolidou, na prática, os principais conceitos da disciplina: arquitetura cliente-servidor, API REST, protocolo HTTP, middleware, transparência, persistência, serviços em nuvem e **integração com um webservice externo** (gateway de pagamento) com comunicação síncrona e assíncrona (*webhook*). Além do aprendizado técnico, o projeto evidencia como uma solução simples pode trazer organização e eficiência ao dia a dia de pequenos empreendedores.

O projeto também já está **preparado para o deploy** da própria aplicação em nuvem (contêiner Docker e *blueprint* do Render), somando-se ao banco PostgreSQL já hospedado. Como evolução futura, prevê-se: notificações ao cliente a cada mudança de status, a **paginação** das listagens, o

processamento **assíncrono** do pagamento e a construção de um aplicativo móvel consumindo a mesma API.

Agradecimentos

Agradecemos aos professores e colegas da disciplina de Sistemas Distribuídos e Mobile pelo suporte e orientação ao longo do desenvolvimento do projeto.

Referências

- [1] FIELDING, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Tese (Doutorado) — University of California, Irvine, 2000.
- [2] TANENBAUM, A. S.; VAN STEEN, M. *Sistemas Distribuídos: Princípios e Paradigmas*. 2. ed. São Paulo: Pearson, 2007.
- [3] DEITEL, P.; DEITEL, H. *Java: Como Programar*. 10. ed. São Paulo: Pearson, 2017.
- [4] WALLS, C. *Spring in Action*. 6. ed. Manning, 2022.
- [5] SPRING. *Spring Boot Reference Documentation*. Disponível em: <https://spring.io/projects/spring-boot>. Acesso em: (data).
- [6] FOWLER, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.